

Surveiller et résoudre un incident technique

Cas étudié : interférence entre deux combats dans un même processus.

SURVEILLER

Métriques, journaux et règles d'alerte du projet.
Distinguer fonctionnement technique et intégrité métier.

REPRODUIRE

Ancien code extrait de Git.
Deux états A et B.
Imbrication déterministe.
Aucun incident de production supposé.

VÉRIFIER

Comparer avant / après.
Identifier le correctif versionné.
Expliquer les limites et les suites opérationnelles.

E5 : 10 minutes ; dossier technique complémentaire joint au support.

Observer le parcours complet du joueur

Django /metrics → Prometheus → Grafana ; journaux de services en console.

APPLICATION

Démarrages, tours et résultats.
Combats et actions.
Délai navigateur jusqu'au retour de choix utilisables.

SERVICES IA

Opérations, tentatives et retries.
Durée logique incluant reprises.
Échecs, sorties invalides et RAG.
Coût estimé si tarifs configurés.

JOURNAUX

Date, niveau, logger, message.
Loggers agents et retrieval.
Erreurs contextualisées.
Contenu à revoir avant partage.

Les métriques évitent les identités ; les messages et traces doivent aussi être contrôlés.

Les alertes techniques ne détectent pas tout

Règles existantes dans `monitoring/prometheus/alerts.yml`.

INDISPONIBILITÉ

Plus de 2 opérations indisponibles sur 10 min.
Condition maintenue 2 min.
Signal pour diagnostic fournisseur.

LATENCE / QUALITÉ

p95 > 30 s sur fenêtre 15 min, pendant 10 min.
Sortie invalide : fenêtre 15 min, maintien 1 min.

ANGLE MORT MÉTIER

Un combat peut renvoyer HTTP 200 tout en modifiant le mauvais état.
Aucune règle actuelle ne détecte cet invariant.

Le récepteur Alertmanager est vide : aucune réception externe démontrée.

L'ancienne version attaque le mauvais monstre

Scénario contrôlé : chaque monstre démarre à 8 PV ; dégâts fixés à 3.

1 / RESTAURER A

Le processus charge le joueur A et son monstre dans des variables globales.

2 / RESTAURER B

Avant l'attaque de A, B charge son combat.
Les mêmes variables globales pointent maintenant vers B.

3 / REPRENDRE A

L'action destinée à A utilise l'état global de B.
Monstre A : 8 PV.
Monstre B : 5 PV.

Attendu : A perd 3 PV ; observé avant correctif : B perd 3 PV.

Un état global partage une identité implicite

La sélection du combat dépendait du dernier appel de restauration.

AVANT

player et current_monster au niveau du module.
restore_combat remplace leurs références.
player_action les lit implicitement.

MÉCANISME

Chaque session web possède ses données, mais la fonction métier réutilise une mémoire de processus partagée.

CONSÉQUENCE

Une interruption entre restauration et résolution peut cibler un autre combat.
Les compteurs HTTP ne voient pas cette erreur métier.

L'isolement de la session web doit être conservé jusque dans les fonctions métier.

Passer le combat explicitement aux fonctions

Correctif versionné le 3 septembre 2026 : commit 8a3cf39.

OBJET DÉDIÉ

CombatSession regroupe joueur, monstre, journal et défense.
Une instance liée à l'état traité.

APPELS EXPLICITES

```
restore_combat_session(...)  
↓  
resolve_player_action(session,  
action)  
↓  
resolve_monster_attack(session)
```

MISE À JOUR LOCALE

GameEngine récupère joueur et monstre dans cette instance, puis met à jour son état de partie.

La résolution du combat Django ne dépend plus du dernier combat restauré globalement.

Le même scénario touche désormais le bon état

Résultats vérifiés le 14 septembre 2026.

AVANT CORRECTIF

Monstre A : 8 → 8 PV.
Monstre B : 8 → 5 PV.

Le mauvais combat est modifié.

CODE ACTUEL

Monstre A : 8 → 5 PV.
Monstre B : 8 → 8 PV.

L'action reste dans la session A.

RÉGRESSION

Test d'isolation Django réussi.
44 tests applicatifs ciblés réussis.
Cas absent / invalide et reprise de combat également exercés.

L'imbrication est prouvée ; la montée en charge et les requêtes simultanées restent à tester.

Relier le correctif à l'exploitation

Une résolution documentée et des actions opérationnelles encore à compléter.

PREUVES DISPONIBLES

Scénario, cause et correctif.
Commit historique identifié.
Reproduction avant / après.
Test de non-régression et commandes.

À JOINDRE

Ticket dans l'outil de suivi.
Chronologie de découverte réelle.
Alerte reçue en environnement test.
Déploiement du correctif et contrôle après livraison.

PRÉVENIR

Tester les invariants métier.
Revoir les autres états partagés.
Tester concurrence et charge.
Mettre à jour le guide de diagnostic.

Le dossier E5 distingue les faits historiques, la reconstruction et les actions proposées.

Rejouer les preuves sans toucher au jeu

Depuis la racine du dépôt ; aucune génération réelle requise.

AVANT / APRÈS

```
uv run python  
certif/presentation-bloc3/  
reproduce_incident.py
```

Ancien code lu via Git.

TEST D'ISOLATION

```
uv run python  
src/django/manage.py test  
game.tests.CombatEngineTests  
--noinput
```

MIGRATIONS

```
uv run python  
src/django/manage.py  
makemigrations --check  
--dry-run
```

Les commandes exactes et le lot complet figurent dans [preuves/verification.md](#).

Retrouver les pièces du dossier

Documentation applicative sous docs/block3/.

CADRAGE / RÉALISATION

C14 : besoins et traçabilité.
C15 : modèles et décisions.
C16 : backlog et revues.
C17 : code, qualité et sécurité.

QUALITÉ / LIVRAISON

C18 : tests et workflow CI.
C19 : Dockerfile, artefact et
procédure de promotion.
Preuves distantes à ajouter.

EXPLOITATION

C20 : monitoring, règles et logs.
C21 : dossier d'incident, correctif
Git et reproduction.
Ticket et déploiement à ajouter.

Les statuts des anciennes notes peuvent être en retard sur le code : dater les preuves.

Expliquer les frontières de la preuve

Trois questions probables du jury.

EST-CE DÉPLOYÉ ?

Le packaging et la procédure existent. Le job staging reste à compléter ; aucune mise en production prouvée ici.

EST-CE ACCESSIBLE ?

Des améliorations et tests de structure existent. Clavier, contraste, zoom et lecteur d'écran restent à auditer.

INCIDENT DÉTECTÉ COMMENT ?

Défaut historique reconstruit par imbrication contrôlée. Aucune alerte de production ni détection automatique de ce défaut n'est revendiquée.

Présenter exactement ce qui a été observé, testé et livré.